

```

1  (*
2      CS51 Lab 19
3      2-dimensional cellular automata
4  *)
5  (*
6      SOLUTION
7  *)
8
9  module G = Graphics ;;
10
11  (*****
12  Do not change either of the two module type signatures in this
13  file. Doing so will likely cause your code not to compile
14  against our unit tests.
15  *****)
16
17  (*.....
18  Specifying automata
19
20  A cellular automaton (CA) is a square grid of cells each in a
21  specific state. The grid is destructively updated according to an
22  update rule. The evolving state of the CA can be visualized by
23  displaying the grid in a graphics window.
24
25  In this implementation, a particular kind of automaton, with its
26  state space and update function, is specified by a module satisfying
27  the `AUT_SPEC` signature. *)
28
29  module type AUT_SPEC =
30  sig
31      (* Automaton parameters *)
32      type state (* the space of possible cell states *)
33      val initial : state (* the initial (default) cell state *)
34      val grid_size : int (* width and height of grid in cells *)
35      val update : (state array array) -> int -> int -> state
36                      (* update grid i j -- returns the new
37                      state for the cell at position `i, j`
38                      in `grid` *)
39      val random_state : unit -> state
40                      (* returns a random state *)
41      (* Rendering parameters *)
42      val name : string (* a display name for the automaton *)
43      val side_size : int (* width and height of cells in pixels *)
44      val cell_color : state -> G.color
45                      (* color for each state *)
46      val legend_color : G.color (* color to render legend *)
47      val font : string option (* optional font for legend as per
48                               Graphics module *)
49      val render_frequency : int (* how frequently grid is rendered
50                               (in ticks) *)
51  end ;;
52

```

```

53 (*.....
54 Automata functionality
55
56 Implementations of cellular automata provides for the following
57 functionality:
58
59 * A current (mutable) grid that can be modified (`replace_grid`).
60
61 * Creating additional fresh grids (`fresh_grid`).
62
63 * Initializing the graphics window in preparation for rendering
64 (`graphics_init`).
65
66 * Mapping the automaton's update function simultaneously over all
67 cells of the current grid (`update_grid`).
68
69 * Running the automaton using a particular update function, and
70 rendering it as it evolves (`run_grid`).
71
72 as codified in the `AUTOMATON` signature.
73
74 Note the difference between the argument structure of functions in
75 `life.ml`, where the grid is passed in as argument to several
76 functions, as compared to the approach here, where there is a single
77 mutable current grid that is updated by the functions, so that no
78 grid argument is required.
79 *)
80
81 module type AUTOMATON =
82   sig
83     (* state -- Possible states that a grid cell can be in *)
84     type state
85
86     (* grid -- 2D grid of cell states *)
87     type grid = state array array
88
89     (* current_grid -- The current grid of the appropriate size as per
90        the spec, which is initialized with all cells being in the
91        initial state. It can be modified directly or by `replace_grid`
92        or updated with the automaton's update rule. *)
93     val current_grid : grid
94
95     (* fresh_grid () -- Returns a fresh grid of the appropriate size
96        as per the spec, initialized with all cells being in the
97        initial state. *)
98     val fresh_grid : unit -> grid
99
100    (* random_grid () -- Returns a fresh grid of the appropriate
101       size as per the spec, initialized with cells in random
102       states. *)
103    val random_grid : unit -> grid
104
105    (* replace_grid new_grid -- Destructively changes the current grid
106       (`current_grid`, above) to `new_grid`. *)

```

```

107     val replace_grid : grid -> unit
108
109     (* graphics_init () -- Initialize the graphics window to the
110        correct size and other parameters. Auto-synchronizing is off,
111        so our code is responsible for flushing to the screen upon
112        rendering. *)
113     val graphics_init : unit -> unit
114
115     (* update_grid () -- Updates the current grid one "tick" by
116        updating each cell simultaneously as per the CA's update
117        rule. *)
118     val update_grid : unit -> unit
119
120     (* run_grid grid -- Initializes the current grid with `grid` and
121        repeatedly updates it using the automaton's update rule,
122        rendering the grid state to the graphics window until a key is
123        pressed. Assumes graphics has been initialized with
124        `graphics_init`. *)
125     val run_grid : grid -> unit
126 end ;;
127
128 (* .....
129    Implementing automata based on a specification
130
131    Given an automaton specification (satisfying `AUT_SPEC`), the
132    `Automaton` functor delivers a module satisfying `AUTOMATON` that
133    implements the specified automaton. *)
134
135 module Automaton (Spec : AUT_SPEC)
136   : (AUTOMATON with type state = Spec.state) =
137   struct
138     type state = Spec.state
139     type grid = Spec.state array array
140
141     (* fresh_grid () -- See module type documentation *)
142     let fresh_grid () : unit : grid =
143       Array.make_matrix Spec.grid_size Spec.grid_size Spec.initial
144
145     (* current -- The current grid, which is destructively updated
146        after each tick; initialized with the initial state. *)
147     let current_grid =
148       fresh_grid ()
149
150     let random_grid () : grid =
151       let mat = fresh_grid () in
152       for i = 0 to Spec.grid_size - 1 do
153         for j = 0 to Spec.grid_size - 1 do
154           mat.(i).(j) <- Spec.random_state ()
155         done
156       done;
157       mat ;;
158
159     (* replace_grid -- See module type documentation *)
160     let replace_grid (new_grid : grid) : unit =

```

```

161   for i = 0 to Spec.grid_size - 1 do
162       for j = 0 to Spec.grid_size - 1 do
163           current_grid.(i).(j) <- new_grid.(i).(j)
164       done
165   done
166
167   (* graphics_init -- See module type documentation *)
168   let graphics_init () : unit =
169       G.open_graph "";
170       G.resize_window (Spec.grid_size * Spec.side_size)
171                       (Spec.grid_size * Spec.side_size);
172       (match Spec.font with
173       | None -> ()
174       | Some fontspec -> G.set_font fontspec);
175       G.auto_synchronize false
176
177   (* update_grid -- See module type definition. *)
178   let update_grid =
179       (* use a single static temp grid across invocations *)
180       let temp_grid = fresh_grid () in
181       fun () -> for i = 0 to Spec.grid_size - 1 do
182                   for j = 0 to Spec.grid_size - 1 do
183                       temp_grid.(i).(j) <- Spec.update current_grid i j
184                   done
185               done;
186               replace_grid temp_grid
187
188   (* render_grid grid legend -- Renders the `grid` to the already
189      initialized graphics window including the textual `legend` *)
190   let render_grid (grid : grid) (legend : string) : unit =
191       (* draw the grid of cells *)
192       for i = 0 to Spec.grid_size - 1 do
193           for j = 0 to Spec.grid_size - 1 do
194               G.set_color (Spec.cell_color grid.(i).(j));
195               G.fill_rect (i * Spec.side_size) (j * Spec.side_size)
196                           (Spec.side_size - 1) (Spec.side_size - 1)
197           done
198       done;
199       (* draw the legend *)
200       G.moveto (Spec.side_size * 2) (Spec.side_size * 2);
201       G.set_color Spec.legend_color;
202       G.draw_string legend;
203       (* flush to the screen *)
204       G.synchronize ()
205
206   (* run_grid -- See module type documentation *)
207   let run_grid (initial_grid : grid) : unit =
208       replace_grid initial_grid;
209       let tick = ref 0 in
210       render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
211       ignore (G.read_key ()); (* pause at start until key is pressed *)
212       while not (G.key_pressed ()) do
213           update_grid ();

```

```

214         tick := succ !tick;
215         if !tick mod Spec.render_frequency = 0 then
216             render_grid current_grid (Printf.sprintf "%s: tick %d" Spec.name !tick);
217         done;
218         ignore (G.read_key ()); (* clear the keypress *)
219         ignore (G.read_key ()); (* await another keypress *)
220         G.close_graph () ;;
221     end ;;

```